

Technical Assignment: Healthcare / MedTech Robotics**Candidate:** Gokul Soman**Duration:** 2 days**Submission format:** GitHub repository or zipped project folder with README, code, screenshots/video, and results.

This assignment is designed to evaluate your competencies in robotics simulation, perception AI, medical/healthcare problem framing, and practical engineering execution. Your resume shows experience in ROS/ROS2, Gazebo, SolidWorks/Fusion360, computer vision, model optimization, embedded deployment, and exoskeleton-related mechanical design, so the tasks below are aligned to those strengths.

You may choose **one of the two assignments** below. If time permits, you may add a small extension from the second topic, but a well-executed single assignment is preferred over two incomplete ones.

Option 1: Robotic Arm Based Femoral Artery Catheterization Simulation**Objective**

Build a simplified ROS2/Gazebo-based simulation of a robotic arm assisting in femoral artery catheterization. The system should detect or localize a target vessel region from ultrasound-like/public medical image data and use that output to guide a robotic arm toward a planned access point.

This is not expected to be clinically accurate. The goal is to evaluate your ability to simplify a healthcare robotics problem, build a working prototype, and clearly document assumptions and limitations.

Expected Scope

You are expected to build a minimal working prototype with the following components:

1. ROS2 robotic simulation

- Use ROS2 and Gazebo.
- Use any available robotic arm model or a simple custom arm.
- The end-effector can represent either:
 - an ultrasound probe, or
 - a needle/catheter guide.

2. Medical image perception

- Use publicly available ultrasound or vascular image data.
- Implement one of the following:
 - YOLO/CNN-based vessel localization,
 - segmentation-based artery detection,
 - or a simpler classical computer vision baseline if justified.
- The model does not need to be highly accurate, but the pipeline should be functional and explainable.

3. ROS2 perception node

- Create a ROS2 node that takes an image input.
- Outputs the detected vessel/target location as:
 - bounding box,
 - segmentation mask,
 - center point,
 - or target coordinates.

4. Robot target planning

- Convert the perception output into a target pose or simplified access point.
- Move the robotic arm/end-effector toward this target in simulation.
- You may use MoveIt2 if comfortable, or a simplified control/motion approach.

5. Documentation

- Explain your assumptions clearly.

- Mention what is simplified compared to a real femoral catheterization system.
- Discuss clinical and engineering risks such as:
 - ultrasound calibration,
 - patient variability,
 - vessel movement,
 - sterility,
 - force control,
 - safety interlocks,
 - false detection risk.

Suggested Dataset Direction

You may use any public vascular ultrasound or related medical image dataset. If femoral artery-specific data is unavailable, you may use another vessel ultrasound dataset as a proxy, but clearly state the limitation.

Deliverables

Submit the following:

1. GitHub repository or zip file.
2. README with:
 - setup instructions,
 - system architecture,
 - assumptions,
 - dataset used,
 - model/perception approach,
 - limitations,
 - future improvements.
3. ROS2 launch file or clear run instructions.
4. Screenshots or short demo video showing:
 - image/vessel detection,
 - target generation,
 - robotic arm movement in simulation.
5. Code for:
 - perception pipeline,
 - ROS2 node,
 - robot simulation/control.
6. Optional:
 - Dockerfile,
 - trained model weights,
 - evaluation metrics,
 - architecture diagram.

Evaluation Criteria

Area	Weight
ROS2/Gazebo integration	25%
Perception/modeling approach	25%
Robotic motion/target planning	20%
Healthcare/clinical reasoning	15%
Code quality, reproducibility, README	15%

What We Are Looking For

A strong submission will show:

- a working end-to-end demo, even if simplified;

- clean ROS2 node structure;
- sensible perception model or baseline;
- practical understanding of robotics constraints;
- awareness of healthcare safety and validation requirements;
- clear documentation of what works and what does not.

Option 2: Exoskeleton Gait Intent Recognition and Assistive Control Prototype

Objective

Build a small prototype for an assistive lower-limb exoskeleton system that detects user gait intent or locomotion mode from public wearable sensor data and maps the prediction to a simple assistive control state.

The goal is to evaluate your ability to work with biomechanical time-series data, build an ML pipeline, and think through how an exoskeleton control system would be structured.

Expected Scope

You are expected to build a minimal working prototype with the following components:

- Public dataset usage**
 - Use a public dataset containing one or more of:
 - IMU data,
 - EMG data,
 - joint angle data,
 - gait phase labels,
 - locomotion mode labels,
 - exoskeleton or wearable sensor signals.
- Gait mode or phase classification**
 - Build a model to classify one of the following:
 - walking vs standing,
 - stair ascent/descent,
 - ramp ascent/descent,
 - gait phase,
 - sit-to-stand transition,
 - or any similar locomotion category available in the dataset.
 - Acceptable models include:
 - Random Forest,
 - SVM,
 - 1D CNN,
 - LSTM/GRU,
 - lightweight Transformer,
 - or another justified approach.
- Signal processing**
 - Perform basic preprocessing:
 - filtering,
 - windowing,
 - normalization,
 - train/test split,
 - feature extraction if needed.
- Assistive control logic**
 - Create a simple finite-state machine or rule-based controller.
 - Example:
 - if mode = stair ascent → increase knee assistance state,
 - if mode = level walking → normal gait assistance,

- if mode = standing → no assistive torque.
 - You do not need to simulate real torque accurately. A clear control concept is enough.
- 5. **Optional ROS2 integration**
 - Wrap the classifier or controller as a simple ROS2 node.
 - Publish predicted gait mode and assistive state.
- 6. **Documentation**
 - Explain dataset choice.
 - Explain model choice.
 - Discuss latency, safety, false predictions, patient variability, and deployment considerations.

Deliverables

Submit the following:

1. GitHub repository or zip file.
2. README with:
 - dataset link/source,
 - setup instructions,
 - preprocessing steps,
 - model architecture or algorithm,
 - control logic,
 - results,
 - limitations.
3. Python scripts or notebook for:
 - data loading,
 - preprocessing,
 - model training/inference,
 - evaluation.
4. Results:
 - accuracy, F1 score, or confusion matrix;
 - sample prediction plot;
 - explanation of latency or real-time feasibility.
5. Assistive control logic:
 - finite-state-machine diagram, table, or code.
6. Optional:
 - ROS2 node,
 - ONNX/TFLite export,
 - Dockerfile,
 - short demo video.

Evaluation Criteria

Area	Weight
Data preprocessing and signal handling	25%
ML model design and evaluation	25%
Exoskeleton control logic	20%
MedTech/rehab safety reasoning	15%
Code quality, reproducibility, README	15%

What We Are Looking For

A strong submission will show:

- practical handling of time-series sensor data;
- clear model selection and evaluation;

- an understandable assistive-control concept;
- awareness of real-time and safety constraints;
- clean, reproducible code;
- ability to explain trade-offs.

Final Submission Instructions

Please submit your work within **2 days**.

Your submission should include:

1. Source code.
2. README with setup and run instructions.
3. Screenshots or short video demonstrating the result.
4. Short technical explanation of your design choices.
5. Known limitations and next steps.

You are encouraged to make reasonable simplifications. The focus is not on building a production-grade medical device, but on demonstrating your ability to create a technically sound, explainable, and reproducible prototype in a healthcare/medtech robotics context.